Lab: 10 MCMC

To implement the Metropolis-Hasting algorithm in Python to generate samples from a target probability distribution using room weights and acceptance probabilities.

Step 1: Define rooms and weights

rooms = ["A", "B", "C"]

weights = {"A": 1, "B": 2, "C": 3}

rooms store the three room names.

weights assign each room its attraction weight.

Step 2: Calculate Target Probabilities

total = sum(weights.values())

target = {r: weights[r] / total for r in rooms}

total = 1+2+3 = 6

The target probabilities are A = 1/6, B = 2/6 & C = 3/6

Step 3: Initialize the current room and Samples

current = "A"

samples = [ ]

The visitor starts in room A

Samples store the room visited after every decision including rejected moves.

Step 4: Define the trials

trials = [("C", .80), ("A", .50), ("B", .60),
("A", .20), ("C", .70), ("B", .90),
("A", .40), ("C", .10), ("B", .50),
("A", .90), ("B", .30), ("C", .20)]

Each tuple contains the proposed room and random value u. These fixed values makes the dry run reproducible.

Step 5: Process each trial

for step, (proposed, u) in enumerate (trials, 1):

enumerate (trials, 1) numbers the trials from 1 to 12

proposed stores the proposed room.

u stores the random value.

Step 6: Get the room weights

old_weight = weights [current]

new_weight = weights [proposed]

old_weight is the weight of current room.

new_weight is the weight of the proposed room.

Step 7: Calculate acceptance probability

ratio = new_weight / old_weight

alpha = min (1.0, ratio)

ratio compares the proposed room's weight with current room's weight

alpha limits the acceptance probability to a max of 1.

Step 8: Accept or reject the proposal

if u < alpha:

    current = proposed

if u is less than alpha, the visitor moves to the proposed room. Otherwise, the visitor stays in the current room.

Step 9: Record the current room

samples.append (current)

The current room is added to samples after every decision, even if the proposal is rejected

Step 10: Count the visits
counts = {r: sample.count(r) for r in rooms}
This counts how many times A, B & C appear in the recorded samples

Step 11: Display the results
print("counts:", counts)
print("Target probabilities:", target)
counts displays the observed visit counts
target displays the desired probabilities for comparision.